

INTERNET OF THINGS

CHALLENGE 3

Node Red

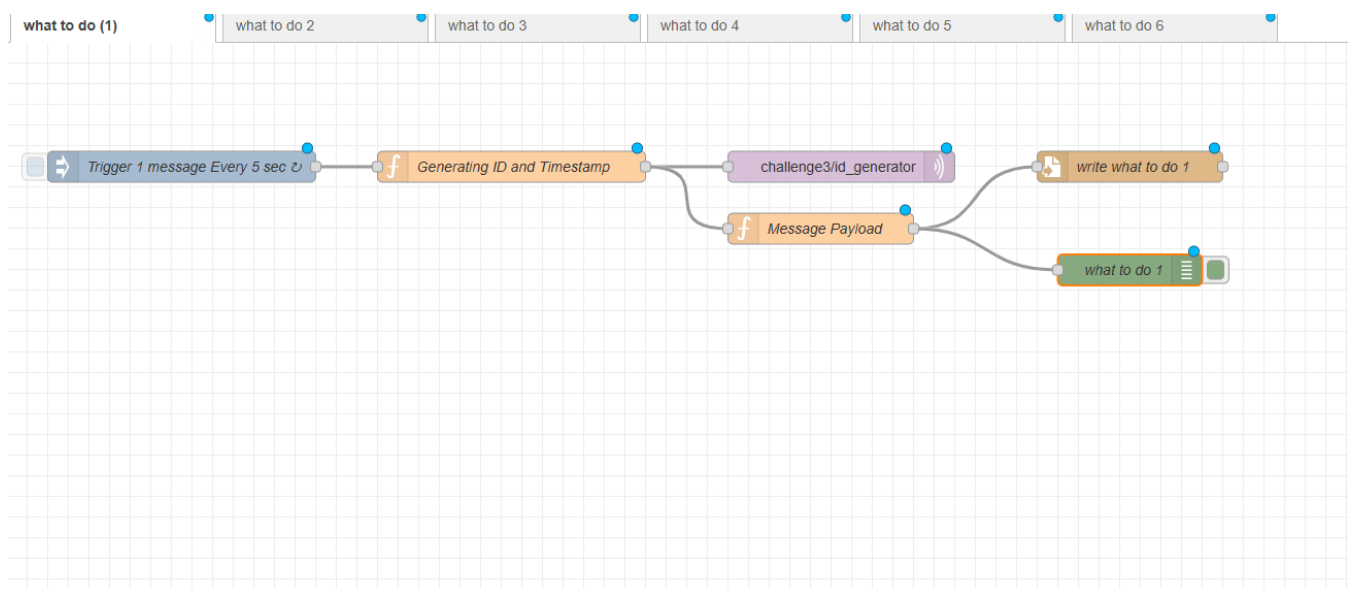
Team Leader : Balasakthi Shanmugaraja – 11033318

Team member : Karuniaa Jacob Selvi – 11034239

First, two terminal windows must be opened. In the first terminal, the Mosquitto broker is started using the command **mosquitto -p 1884 -v**, which launches the broker on port 1884. In the second terminal, **Node-RED** is started by running **node-red**, which opens the node red at **http://localhost:1880** for building and executing the flow.

What to do? (1)

The timestamp node initiates a flow at intervals of 5 seconds, generating the current time using the `Date.now()` function and transmitting it to the designated function node. Within the 'Generating ID and Timestamp' function, a random variable is generated in the range of 0 to 30,000, and both the timestamp and the random variable are merged into a single payload. The resulting message payload adopts the format `{"id": rand_id, "timestamp": time}`.

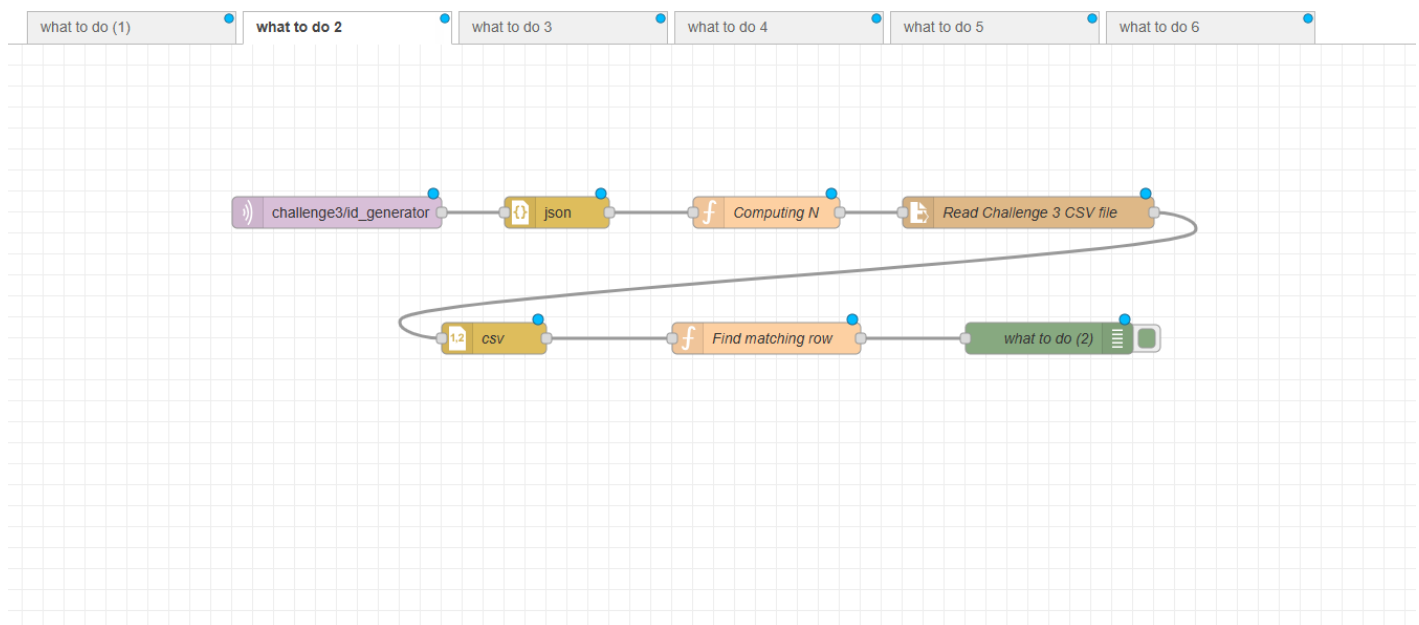


To publish the MQTT messages to the local broker, an MQTT out node is utilized, configured to connect to the server at localhost on port 1884. The MQTT out node is set to publish under the topic "challenge3/id_generator". Consequently, each generated ID and timestamp pair is transmitted to the broker through this channel.

In Same time , the system requires the storage of generated IDs and timestamps in a CSV file, where each row corresponds to a single message. A context variable is established to track the row number, incrementing automatically with each execution of the code. The 'Message Payload' function processes each generated message to format it correctly for storage. Subsequently, a file node is introduced to store the organized messages into a CSV file named 'Id_log.csv'. The data is structured into three columns: Number, ID, and Timestamp, ensuring that numerical values are appropriately parsed and written into the file. A debug node is also connected to visualize each outgoing message for verification purposes.

What to do (2)

The second part of the flow begins when the mqtt in node subscribes to the same MQTT topic as (challenge3/id_generator) receiving each message sent by the publishing flow. These incoming messages are automatically parsed into JSON format using a json node. The parsed messages are then processed by the 'Compute N' function node. Here, the ID from each message is extracted, and a modulo operation is performed in $N = id \% 7711$

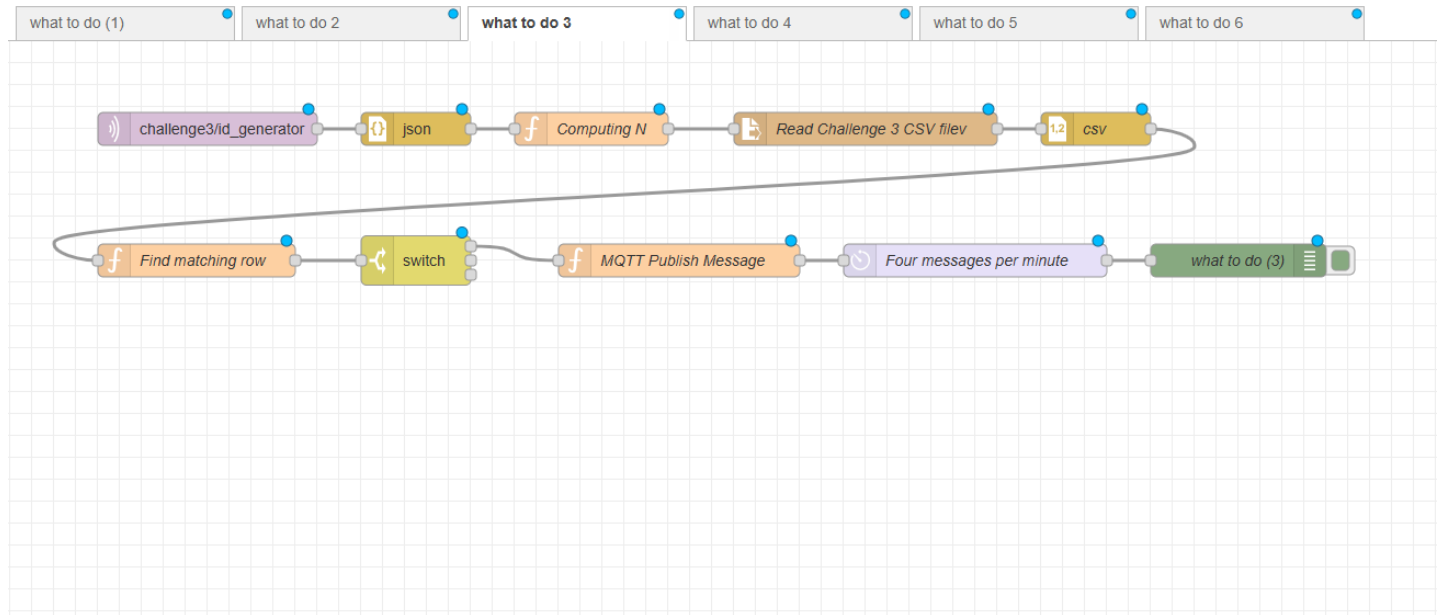


This computed N serves as the reference frame number needed for further matching. The system also maintains a global message counter, incrementing with each new received message. Simultaneously, a file in node reads the content of the challenge3.csv file stored locally. This file contains reference frames with their associated topics, payloads, and message types. The csv node converts the contents of the CSV file into an array of JavaScript objects, where each object represents a row.

The 'Find matching row' function compares the computed N with the 'No.' field in each row of the CSV array. If a matching frame is found (i.e., if $N = \text{Frame No.}$), the corresponding row is forwarded for further processing; otherwise, a warning message is issued in the debug sidebar. Thus, the flow aligns each received ID with its corresponding frame in the challenge3.csv file, enabling accurate message reconstruction based on the reference data.

What to do (3)

After successfully matching a frame from challenge3.csv, the flow uses a switch node to analyze the Info field of the matched frame.



If the Info field contains the phrase "Publish Message", the flow proceeds to the 'Process Publish Message' function node. In this node, the system extracts all MQTT topics embedded within the frame's Info field, using regular expressions to correctly parse multiple topics, if present.

Simultaneously, the Payload field of the frame, which may contain multiple JSON objects concatenated together, is parsed. The system splits the payload into separate valid JSON objects representing individual messages.

The extracted topics are then matched one-to-one with the parsed payloads, creating a structured array of publish messages where each message contains:

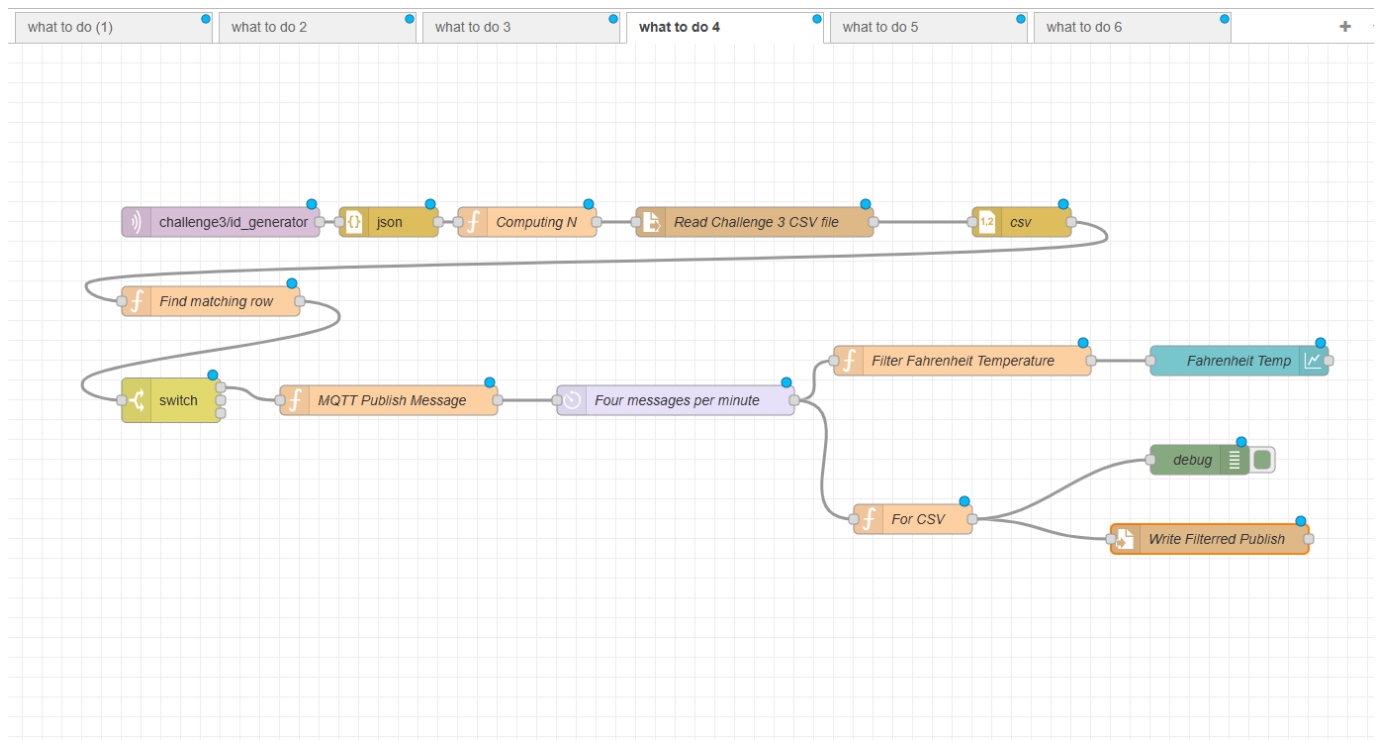
- Topic
- Payload
- Timestamp
- ID

These publish messages are then passed through a delay node configured to rate limit outgoing traffic to a maximum of four messages per minute, ensuring that message flow remains compliant with low-power wide-area network (LPWAN) constraints.

This phase reconstructs the original intended publish messages from the CSV reference frames and prepares them for further filtering.

What to do (4)

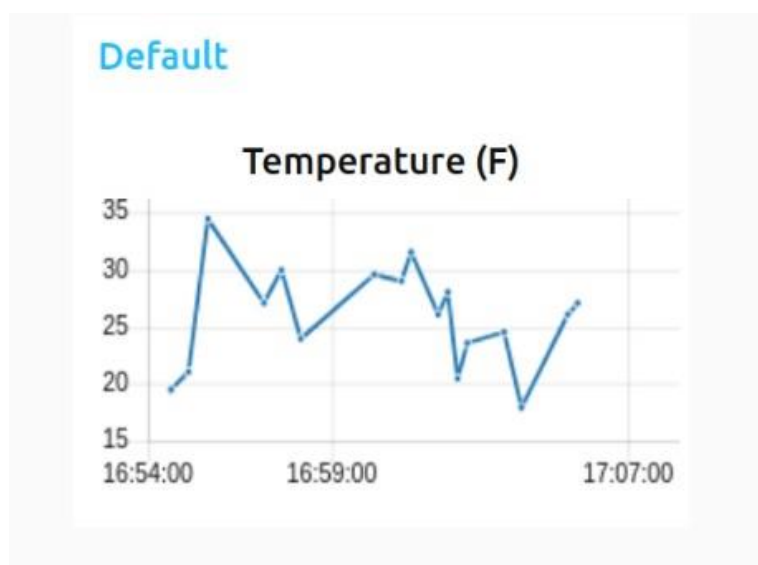
The structured array of publish messages generated in the previous step enters the 'Filter Fahrenheit Temperature' function



Here, each publish message's payload is analyzed individually to detect messages that meet the following:

- type field equals "temperature"
- unit field equals "F"

If both conditions are satisfied, the range field (an array of two values: minimum and maximum readings) is accessed.



The mean temperature value is sent to a Node-RED dashboard chart node, plotting temperature readings in real-time, offering a visual insight into the environment being simulated.

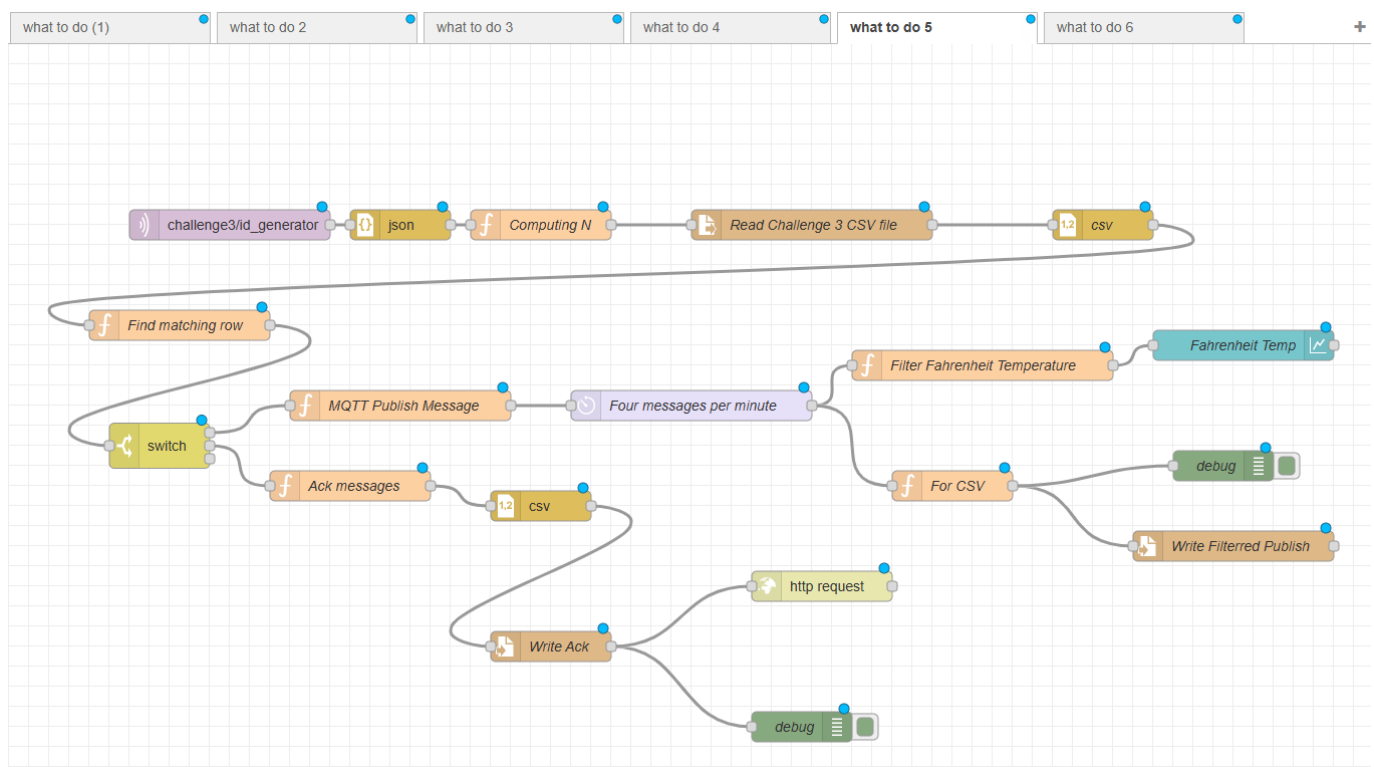
Simultaneously, another function node prepares structured CSV rows for each valid Fahrenheit temperature message. Each row contains:

- Sequential message number
- Longitude
- Latitude
- Mean temperature value
- Message type
- Unit (Fahrenheit)
- Description field

The generated rows are appended into a file named '**filtered_publish.csv**', ensuring that all relevant temperature publish messages are properly archived for analysis.

What to do (5)

If the received frame does not represent a publish message but instead matches an acknowledgment type, the switch node redirects the flow to the 'Process ACK Message' function.



The flow checks if the Info field exactly matches one of the following:

- "Connect Ack"
- "Publish Ack"
- "Subscribe Ack"
- "Unsubscribe Ack"

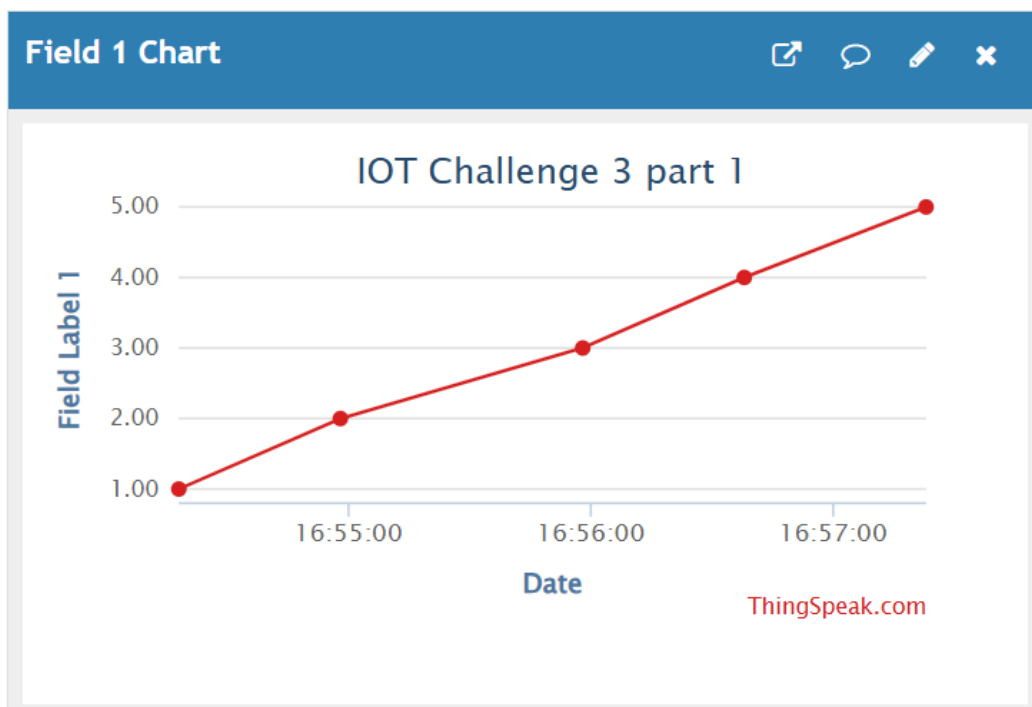
If so, an ACK counter maintained in global context is incremented by one.

A structured CSV line is then created with:

- A sequential ACK number
- Current timestamp (in UNIX time)
- Subscription ID
- Message type

This line is appended to a file named 'ack_log.csv', ensuring a complete record of acknowledgment events during flow execution.

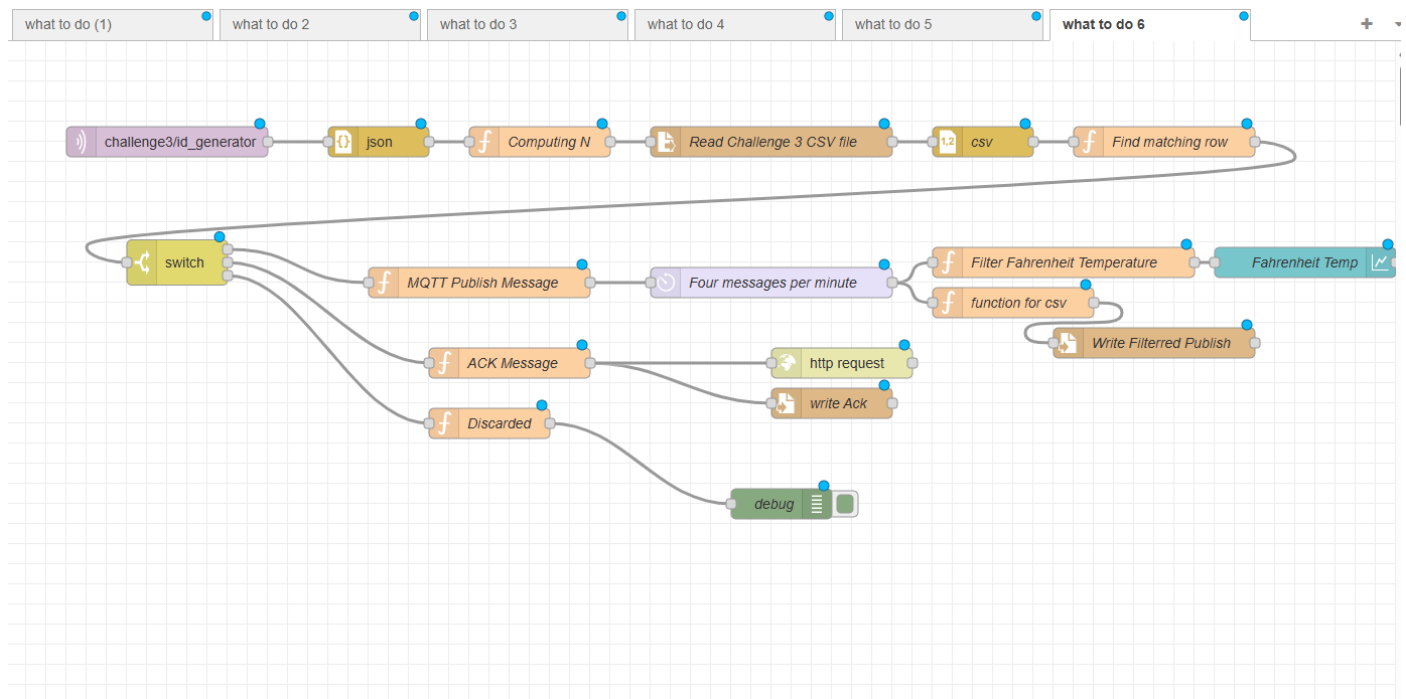
Additionally, for every new ACK detected, an HTTP GET request is made to ThingSpeak's API, updating a public channel field to reflect the current total number of received acknowledgments.



This dual recording ensures both local and remote tracking of message acknowledgments.

What to do (6)

Any message that does not meet the criteria for either a Publish Message or a valid Acknowledgment is categorized as a discarded message.



These messages are processed by the 'Discarded msgs' function, where a discarded counter is incremented using a global context variable. Throughout the operation, the system simultaneously tracks the total number of received ID messages (whether processed as publish, ack, or discarded).

Once the total number of messages reaches 80, corresponding to the project requirement, the flow automatically stops further processing. A debug node outputs the final count of received and discarded messages, serving as confirmation that the maximum allowed messages have been handled correctly.

This mechanism guarantees that the flow remains controlled and does not exceed the defined message limit of the challenge.